چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۳۰

طراحی سیستم موبایل و یکپارچه‌سازی — از نیازمندی‌ها تا trade-offها



اسماعیل مرادی

@esmaeeil-moradi700

چرا طراحی سیستم، روز پایانی است؟

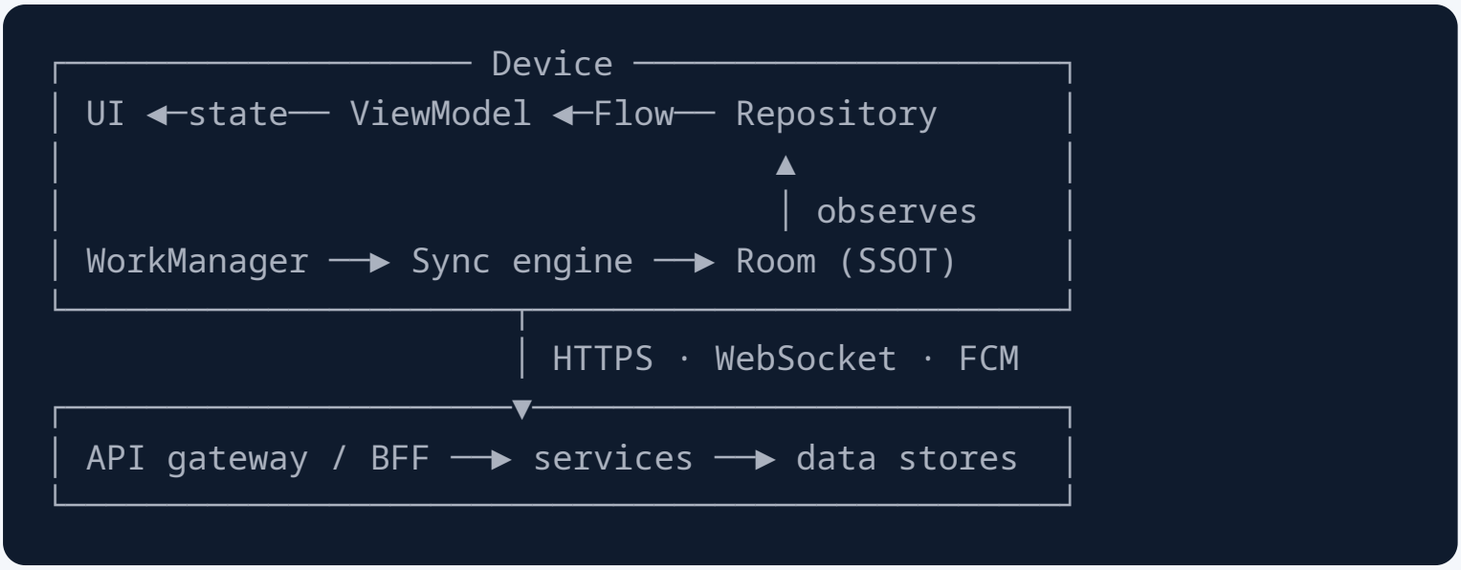
مصاحبه‌های سینیور و آرکیتکت معمولاً همین‌جا تعیین تکلیف می‌شوند. کسی از شما نمی‌خواهد یک API را از حفظ بگویید — یک فیچر را سر تا ته طراحی می‌کنید و از تک‌تک trade-offها دفاع می‌کنید:

- طراحی موبایل، طراحی بک‌اند **نیست**: دستگاه آفلاین می‌شود، kill می‌شود، ساعتش اشتباه است و سال‌ها کد قدیمی اجرا می‌کند
- جزئیات یکپارچه‌سازی — retry، idempotency، سازگاری داده — همان‌جایی است که باگ‌های واقعی production زندگی می‌کنند
- یک **چارچوب** روشن از یک دیاگرام زیرکانه بهتر است
- همهٔ روزهای ۱ تا ۲۹ در یک جواب کنار هم می‌نشینند

سطح	سؤال‌ها
جونیور	قرارداد API، الگوهای یکپارچه‌سازی در کلاینت، دریافت به‌روزرسانی از سرور
مید	تاب‌آوری، سازگاری و تعارض، idempotency، saga
سینیور	چارچوب طراحی، طراحی اپ چت، طراحی فید offline-first
آرکیتکت	طراحی پرداخت، gateway و BFF و نسخه‌بندی، مقیاس و observability و trade-offها

اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: دو نیمه و یک قرارداد



- **دستگاه** یک نود غیرقابل‌اعتماد در یک سیستم توزیع‌شده است — برای همین شرایط طراحی کنید
- داخل اپ، داده یک‌طرفه جریان دارد: از شبکه به Room، از Room با Flow به UI
- **قرارداد** (API و eventها) درز میان این دو نیمه است

وقت مصاحبه را کجا خرج کنید

در مصاحبهٔ موبایل، بک‌اند یک جعبه پشت یک قرارداد است. در سمت دستگاه عمیق شوید: ذخیره‌سازی، همگام‌سازی (sync)، state و حالت‌های خرابی.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

قرارداد API، الگوهای یکپارچه‌سازی در کلاینت، دریافت به‌روزرسانی از سرور



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. یکپارچه شدن با یک بک‌اند پیچیده: اول قرارداد

بک‌اند را یک سیستم بیرونی پشت یک **قرارداد** ببینید، و همهٔ دانش مربوط به آن را داخل **لایهٔ data** نگه دارید. یک قرارداد خوب این‌ها را تعریف می‌کند:

- منابع، عملیات و schemaها — در یک OpenAPI spec، یک GraphQL schema یا یک فایل `.proto`
- یک **مدل خطای تایپ‌دار**، نه فقط یک HTTP status و یک پیام متنی
- صفحه‌بندی با cursor، احراز هویت و refresh توکن، محدودیت نرخ (`429` همراه با `Retry-After`)
- idempotency برای نوشتن‌ها، نسخه‌بندی و سیاست منسوخ‌سازی (deprecation)

سبک	نقطهٔ قوت	مراقب باشید
REST با JSON	ساده، کش HTTP و CDN، ابزار همه‌جا	چند رفت‌وبرگشت برای صفحه‌های پیچیده
GraphQL	کلاینت فیلدها را انتخاب می‌کند؛ یک درخواست برای هر صفحه	کش HTTP سخت‌تر است؛ هزینهٔ کوئری روی سرور
gRPC	protobuf فشرده، streaming، schema سخت‌گیر	به HTTP/2 نیاز دارد؛ بازرسی و کش سخت‌تر



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

«*GraphQL برای موبایل همیشه کارآمدتر است.*» (نسخهٔ ۱) — over-fetching را حذف می‌کند، اما هزینه را به resolverهای سرور منتقل می‌کند، کش HTTP را سخت‌تر می‌کند، و یک کوئری بدطراحی‌شده به کندی پنج فراخوانی REST است. یک **backend-for-frontend** مبتنی بر REST اغلب همان مزیت «یک فراخوانی برای هر صفحه» را می‌دهد.

تلهٔ مصاحبه

«*SDK آمادهٔ یک فروشنده همیشه از API client خودمان ساده‌تر است.*» — شروعش سریع‌تر است، اما حجم اپ، کار هنگام startup، permissionها و وابستگی به فروشنده را اضافه می‌کند. آن را پشت **interface خودتان** بگذارید تا در تست fake شود و بعداً قابل تعویض باشد.

سؤال تکمیلی

«*دانش بک‌اند کجای اپ زندگی می‌کند؟*» — فقط در data sourceها: endpointها، DTOها، headerها. repositoryها DTO را به مدل domain تبدیل می‌کنند؛ ViewModel هیچ‌وقت یک DTO یا HTTP status code نمی‌بیند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. الگوهای یکپارچه‌سازی در کلاینت، نسخهٔ ۲۰۲۶

```kotlin
fun UserDto.toDomain() = User(
    id = UserId(id),
    name = displayName ?: "Unknown",
)

class ProfileRepository(
    private val users: UserApi,
    private val billing: BillingApi,
) {
    suspend fun profile(id: String): Profile = coroutineScope {
        val user = async { users.get(id).toDomain() }
        val plan = async { billing.plan(id).toDomain() }
        Profile(user.await(), plan.await())
    }
}
```

- **Adapter** یعنی mapperهایی مثل `toDomain()` و wrapperهای دور SDKهای فروشنده؛ **Facade** یعنی repository که چند سرویس را پنهان می‌کند
- با `coroutineScope` شکست یک فراخوانی، دیگری را لغو می‌کند — همان structured concurrency (روز ۴)

تلهٔ مصاحبه

«*برای Retrofit و دیتابیس از الگوی Singleton استفاده کنید.*» (نسخهٔ ۱) — شما **یک نمونه** می‌خواهید، نه **دسترسی سراسری**: بگذارید DI با `@Singleton` در Hilt دامنهٔ `OkHttpClient` و Retrofit را مدیریت کند. یک `object` سراسری وابستگی‌ها را پنهان می‌کند و در تست fake نمی‌شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. دریافت به‌روزرسانی از سرور

سازوکار	کِی	هزینه
polling دوره‌ای	داده به‌ندرت تغییر می‌کند؛ ساده‌ترین راه	درخواست‌های هدررفته؛ تأخیر به اندازهٔ فاصلهٔ polling
SSE یا long polling	stream یک‌طرفه از سرور به کلاینت	یک اتصال باز برای هر کلاینت
WebSocket	دوطرفه، تأخیر کم، اپ در foreground	منطق reconnect و resume؛ مصرف باتری اگر باز بماند
پوش FCM	اپ در background است یا اصلاً اجرا نمی‌شود	بدون تضمین: ممکن است دیر برسد یا ادغام شود؛ payload کوچک

داخل اپ، بخش event-driven ساده است: لایهٔ sync در Room می‌نویسد، و هر صفحه `Flow` مربوط به repository را observe می‌کند. **دیتابیس همان event bus است.**



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

«*از یک Event Bus سراسری استفاده کنید تا اجزا از هم جدا بمانند.*» (نسخهٔ ۱) — در ۲۰۲۶ این یک anti-pattern است: وابستگی نامرئی، state بی‌مالک، نشت lifecycle، و eventهایی که وقتی کسی گوش نمی‌دهد گم می‌شوند. state را از repositoryها به شکل `Flow` منتشر کنید — یک منبع حقیقت.

تلهٔ مصاحبه

«*نوتیفیکیشن پوش، خود داده را می‌رساند.*» — FCM را یک **سیگنال** ببینید: «چیزی عوض شده، sync کن». پیام‌ها ممکن است دیر برسند یا ادغام شوند، و اپی که force stop شده هیچ پیامی دریافت نمی‌کند. حقیقت همیشه از API می‌آید.

سؤال تکمیلی

«*می‌شود WebSocket را در background باز نگه داشت؟*» — سؤال ۹ را ببینید: وقتی UI دیده می‌شود وصل شوید، و وقتی دیده نمی‌شود به FCM تکیه کنید.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

تاب‌آوری، سازگاری و تعارض، idempotency، saga



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. تاب‌آوری: retry درست

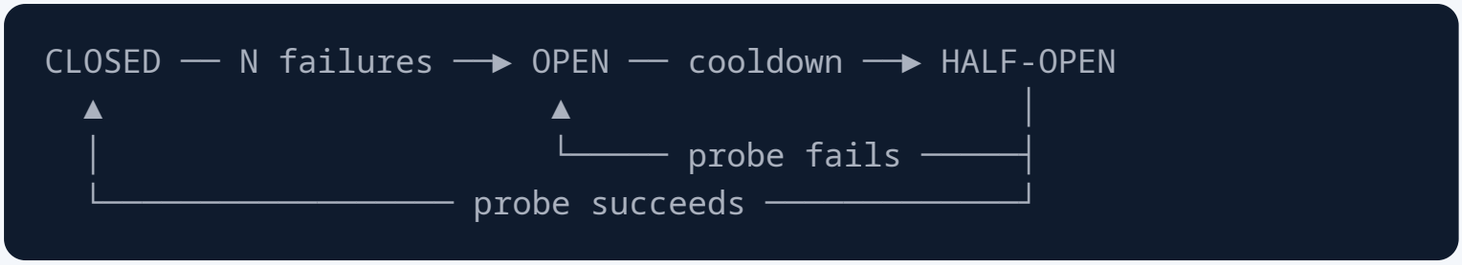
```kotlin
suspend fun <T> withRetry(
    maxAttempts: Int = 4,
    baseDelayMs: Long = 500,
    maxDelayMs: Long = 16_000,
    block: suspend () -> T,
): T {
    var backoff = baseDelayMs
    repeat(maxAttempts - 1) {
        try {
            return block()
        } catch (e: IOException) {
            // transient network error: retry below
        }
        // jitter: wait a random time in [backoff/2, backoff]
        delay(Random.nextLong(backoff / 2, backoff + 1))
        backoff = (backoff * 2).coerceAtMost(maxDelayMs)
    }
    return block() // last attempt: errors propagate
}
```

- فقط خطاهای **گذرا** را retry کنید (خطای I/O، کدهای 5xx، و `429` با رعایت `Retry-After`) و فقط درخواست‌های **امن** را — idempotent، یا دارای idempotency key (سؤال ۶)
- `CancellationException` گرفته نمی‌شود، پس لغو coroutine همچنان کار می‌کند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. circuit breaker و fallback برای ناوگان



- **fallbackها:** دادهٔ کش‌شده با برچسب «به‌روزشده ۵ دقیقه پیش»، پنهان کردن فیچرهای غیرحیاتی، صف کردن نوشتن‌ها در یک outbox
- **اهرم‌های سمت سرور:** هدر `Retry-After`، kill switch با remote config، حداقل نسخهٔ پشتیبانی‌شدهٔ اپ

تلهٔ مصاحبه

«*circuit breaker در کلاینت به سرویسِ خراب فرصت بهبود می‌دهد.*» (نسخهٔ ۱) — یک گوشی تقریباً هیچ اثری ندارد. خطر در **ناوگان** است: میلیون‌ها اپ که بعد از یک قطعی هم‌زمان retry می‌کنند. jitter، `Retry-After` و backoff تحت کنترل سرور از بک‌اند محافظت می‌کنند.

تلهٔ مصاحبه

retry در همهٔ لایه‌ها — OkHttp، یک interceptor، repository و WorkManager. سه لایه با سه تلاش در هر کدام یعنی ۲۷ درخواست برای یک اقدام کاربر. فقط در **یک** جا retry کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. سازگاری داده: UI خوش‌بینانه و تعارض‌ها

بین دستگاه‌ها و سرویس‌ها، داده **در نهایت سازگار** (eventually consistent) است. UI را خوش‌بینانه به‌روز کنید، بعد بگذارید سرور تصمیم بگیرد:

```
suspend fun rename(note: Note, title: String) {
    dao.upsert(note.copy(title = title, sync = PENDING))
    try {
        val saved = api.rename(note.id, title, note.etag)
        dao.upsert(saved.toEntity())          // server copy wins
    } catch (e: HttpException) {
        if (e.code() == 412) dao.markConflict(note.id)
        else dao.upsert(note)                 // roll back
    } catch (e: IOException) {
        outbox.enqueueRename(note.id, title) // retry later
    }
}
```

- `note.etag` به شکل هدر `If-Match` فرستاده می‌شود — یعنی **هم‌روندی خوش‌بینانه** (optimistic concurrency): نوشتنِ کهنه به‌جای بازنویسی بی‌صدا، پاسخ `412 Precondition Failed` می‌گیرد



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. انتخاب راهبرد حل تعارض

راهبرد	چطور	trade-off
آخرین نوشتن برنده است (LWW)	جدیدترین timestamp بازنویسی می‌کند	از دست رفتن بی‌صدای داده؛ ساعت دستگاه‌ها جابه‌جا می‌شود
بررسی نسخه	ETag یا شمارهٔ نسخه؛ نوشتن کهنه رد می‌شود	کلاینت باید ادغام کند یا از کاربر بپرسد
ادغام در سطح فیلد	ادغام ویرایش‌های فیلدهای غیرهم‌پوشان	به ردگیری تغییر برای هر فیلد نیاز دارد
CRDTها	نوع‌داده‌هایی که همیشه ادغام می‌شوند	پیچیدگی و سربار metadata

تلهٔ مصاحبه

«*آخرین نوشتن برنده است: آخرین تغییری که می‌رسد ذخیره می‌شود.*» (نسخهٔ ۱) — ترتیب رسیدن، ترتیب ویرایش نیست: گوشی‌ای که یک روز آفلاین بوده sync می‌کند و روی ویرایش جدیدتر می‌نویسد. از **نسخه‌هایی که سرور تعیین می‌کند** استفاده کنید؛ اگر LWW را انتخاب می‌کنید، بدانید که از دست رفتن داده را انتخاب کرده‌اید.

سؤال تکمیلی

«*UI خوش‌بینانه یا بدبینانه؟*» — خوش‌بینانه برای کارهای ارزان و برگشت‌پذیر (لایک، تغییر نام، ارسال پیام). بدبینانه — نمایش progress و صبر برای سرور — برای پول، کارهای برگشت‌ناپذیر یا امنیتی.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. idempotency: امن کردن retry

متدهای `GET`، `PUT` و `DELETE` طبق تعریف idempotent هستند؛ `POST` و `PATCH` نه. برای نوشتن‌های غیر idempotent یک **idempotency key** بفرستید — یکی برای هر قصد کاربر:

```
interface OrderApi {
    @POST("orders")
    suspend fun create(
        @Header("Idempotency-Key") key: String,
        @Body body: CreateOrderDto,
    ): OrderDto
}

// One key per user intent, persisted with the operation
val op = PendingOrder(key = UUID.randomUUID().toString(), cart)
outboxDao.insert(op)                 // survives process death
api.create(op.key, op.cart.toDto())  // retries reuse op.key
```

سرور نگاشتِ key به اولین نتیجه را ذخیره می‌کند و برای تکرارها همان نتیجه را برمی‌گرداند — یک درخواستِ retryشده نمی‌تواند سفارش دوم بسازد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

«*key را در یک interceptor در OkHttp با یک UUID تازه اضافه می‌کنیم.*» — هر retry یک key **جدید** می‌گیرد، پس سرور آن را درخواست تازه می‌بیند. key باید یک بار برای هر قصد کاربر ساخته شود و همراه عملیات در انتظار ذخیره شود.

تلهٔ مصاحبه

«*صف ما پیام را دقیقاً یک بار تحویل می‌دهد.*» — شبکه یا حداکثر یک بار (at-most-once) تحویل می‌دهد یا حداقل یک بار (at-least-once). **اثر دقیقاً یک‌باره** یعنی تحویل حداقل یک‌باره، به‌علاوهٔ پردازش idempotent و حذف تکراری‌ها در سمت گیرنده.

سؤال تکمیلی

«*idempotency وظیفهٔ کیست؟*» — هر دو طرف. کلاینت key را می‌سازد و ذخیره می‌کند؛ سرور آن را همراه نتیجه نگه می‌دارد و همان key با body متفاوت را رد می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. تراکنش‌های چندمرحله‌ای: saga

```
Order saga — orchestrated by the backend

 step                   compensation if a later step fails
 1 reserve stock        release stock
 2 authorize payment    void the authorization
 3 create shipment      cancel shipment
 4 capture payment      — last step

 step 3 fails → void (2) → release (1) → order FAILED
```

- saga یعنی چند تراکنش محلی، و برای هر کدام یک **جبران** (compensation)؛ بدون قفل سراسری
- **ارکستراسیون** (یک هماهنگ‌کننده) برای پرداخت راحت‌تر قابل مشاهده است؛ **کوریوگرافی** (سرویس‌ها به eventها واکنش نشان می‌دهند) وابستگی کمتری می‌سازد
- ترتیب مهم است: اول رزرو و authorize، و **برداشت پول (capture) در آخر** — نسخهٔ ۱ اول کارت را شارژ می‌کرد و با هر کمبود موجودی مجبور به بازپرداخت می‌شد



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. saga کجا اجرا می‌شود؟

گوشی فقط **یک** فرمان می‌فرستد — `POST /orders` همراه با یک idempotency key — و وضعیت سفارش را با پوش یا polling دنبال می‌کند: از `PENDING` به `CONFIRMED` یا `FAILED`.

تلهٔ مصاحبه

«*اپ اول پرداخت را صدا می‌زند، بعد انبار، بعد ارسال.*» — دستگاه ممکن است بین مراحل kill شود، آفلاین شود یا دستکاری شود. **هرگز کلاینت را هماهنگ‌کنندهٔ** یک تراکنش توزیع‌شده نکنید.

تلهٔ مصاحبه

«*جبران یعنی rollback.*» — جبران یک کار بیزینسی جدید و قابل مشاهده است (بازپرداخت، آزادسازی موجودی، ایمیل عذرخواهی) و گاهی ناممکن. هر مرحله را طوری طراحی کنید که جبرانش وجود داشته باشد — و آن را هم idempotent کنید.

سؤال تکمیلی

«*چرا two-phase commit نه؟*» — مسدودکننده است، هماهنگ‌کننده نقطهٔ شکست واحد است، و ارائه‌دهنده‌های بیرونی (پرداخت، ارسال) در 2PC شما شرکت نمی‌کنند.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

چارچوب طراحی، طراحی اپ چت، طراحی فید offline-first



اسماعیل مرادی

@esmaeeil-moradi700

سؤال ۸. «X را برای اندروید طراحی کن» — چارچوب

#	مرحله	خروجی شما
۱	نیازمندی‌ها	کارکردی، غیرکارکردی، و **خارج از محدوده**
۲	قرارداد API	endpointها و eventها، صفحه‌بندی، خطاها
۳	مدل داده و ذخیره‌سازی	entityها، چه چیزی کجا و تا کی کش می‌شود
۴	sync و آفلاین	مسیر خواندن، مسیر نوشتن، تعارض‌ها
۵	UI state و معماری	لایه‌ها، ماژول‌ها، یک state برای هر صفحه
۶	دغدغه‌های مقیاس	باتری، مصرف داده، حافظه، لیست‌های بزرگ، دستگاه‌های ضعیف
۷	observability	متریک، لاگ، trace، هشدار
۸	trade-offها	چه چیزی را انتخاب کردید، چه چیزی را رد کردید، و چرا

زود رسم کنید

یک دیاگرام جعبه‌ای از لایه‌های کلاینت و بک‌اند در ده دقیقهٔ اول، چیزی به مصاحبه‌گر می‌دهد که به آن اشاره کند — و به شما نقشه‌ای برای بقیهٔ مصاحبه.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. اول نیازمندی‌ها

سؤال‌هایی که ارزش دارد قبل از کشیدن هر چیزی بپرسید:

- کدام کارها باید **آفلاین** کار کنند؟ داده چقدر باید تازه باشد؟
- لحظه‌ای (real-time) یا «در عرض یک دقیقه»؟ پوش وقتی اپ بسته است؟
- یک نتیجهٔ اشتباه یا تکراری چقدر گران تمام می‌شود — لایک، یا پول؟
- مدیا؟ چه حجمی؟ شبکهٔ ضعیف یا دستگاه ضعیف در بازار هدف؟
- حریم خصوصی: PII، رمزنگاری سرتاسری، الزامات قانونی؟

تلهٔ مصاحبه

جواب دادن با زیرساخت بک‌اند — load balancer، sharding، replication — همان کاری که نسخهٔ ۱ کرد. در مصاحبهٔ **موبایل** این عمق اشتباهی است: بک‌اند را در سطح قرارداد نگه دارید مگر اینکه بپرسند، و در کلاینت عمیق شوید.

تلهٔ مصاحبه

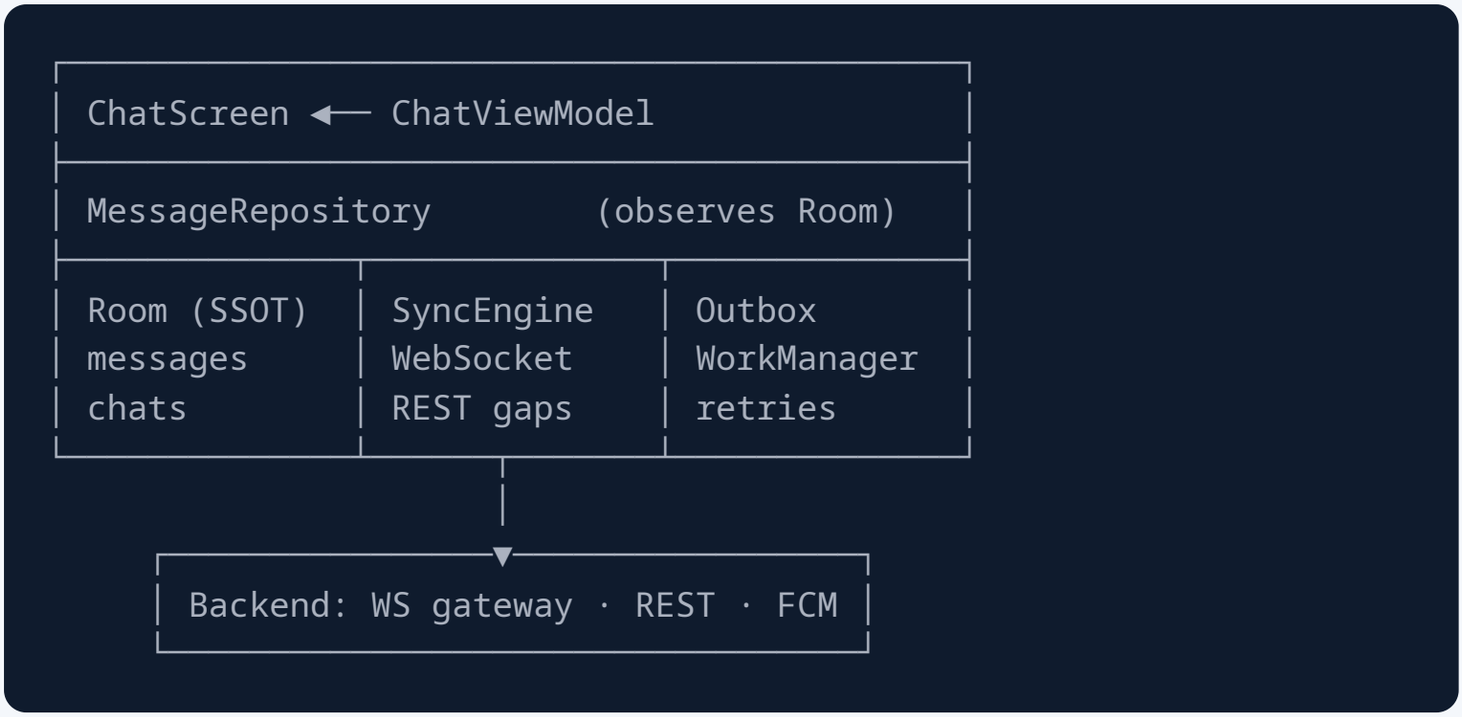
شروع با کتابخانه‌ها — «Retrofit، Hilt، Room». ابزار بعد از نیازمندی‌ها می‌آید؛ مصاحبه‌گر به استدلال نمره می‌دهد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. طراحی اپ چت — نیازمندی‌ها و شکل کلی

کارکردی: چت دونفره و گروهی، متن و تصویر، رسید تحویل و خواندن، تاریخچه، ارسال در حالت آفلاین، نوتیفیکیشن. **غیرکارکردی:** بدون پیام گم‌شده یا تکراری، ترتیب پایدار، آنی در foreground، کم‌مصرف در background. **خارج از محدوده:** تماس صوتی و تصویری، رمزنگاری سرتاسری (سؤال تکمیلی).





اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. قرارداد و مدل داده

```
POST /v1/chats/{id}/messages      body carries clientMsgId
GET  /v1/chats/{id}/messages?beforeSeq=…      history page
GET  /v1/sync?since=<cursor>                  missed events
WS   message.new · message.ack · receipt.read
FCM  data { chatId }  → "go sync", never the message body
```

```
@Entity(
    tableName = "messages",
    indices = [Index(value = ["chatId", "seq"])],
)
data class MessageEntity(
    @PrimaryKey val clientMsgId: String, // UUID from sender
    val serverId: String?,     // null until the server acks
    val chatId: String,
    val seq: Long?,            // server order; null = pending
    val senderId: String,
    val body: String,
    val status: MessageStatus, // PENDING, SENT, READ, FAILED…
    val createdAtLocal: Long,  // display only, never ordering
)
```

کوئری `ORDER BY seq IS NULL, seq, createdAtLocal` پیام‌های تأییدشده را به ترتیب سرور می‌آورد، و پیام‌های در انتظار را در انتها.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. مسیر ارسال و دریافت

ارسال:

۱. یک ردیف `PENDING` با یک `clientMsgId` تازه درج کنید — UI فوراً نشانش می‌دهد
۲. سوکت (یا در حالت آفلاین، worker مربوط به outbox) آن را می‌فرستد؛ `clientMsgId` همان idempotency key است
۳. سرور `seq` را تعیین و ack می‌کند و ردیف `SENT` می‌شود؛ ارسال تکراری همان ack را می‌گیرد

دریافت:

۱. وقتی اپ در foreground است، event از WebSocket می‌رسد و با `clientMsgId` upsert می‌شود (حذف تکراری‌ها مجانی)
۲. شکاف در `seq` (۳۸ را داریم و ۴۱ رسید) با `GET /sync?since=…` پر می‌شود
۳. وقتی اپ در background است، پیام داده‌ای high-priority از FCM، یک sync کوتاه، و بعد نوتیفیکیشن
۴. اتصال دوباره با backoff و jitter؛ ادامه از آخرین cursor

> **تلهٔ مصاحبه**
>
> «*پیام‌ها را با timestamp دستگاه مرتب کنید.*» — ساعت دستگاه‌ها با هم فرق دارد و کاربرها عوضش می‌کنند. بر اساس **sequence سرور** در هر چت مرتب کنید؛ زمان محلی فقط برای نمایش است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. trade-offها و سؤال‌های تکمیلی

تصمیم	انتخاب	ردشده — چرا
لحظه‌ای بودن	WebSocket وقتی UI دیده می‌شود، به‌علاوهٔ FCM	polling، به خاطر تأخیر و مصرف باتری
تحویل	حداقل یک‌بار، به‌علاوهٔ حذف تکراری‌ها	«دقیقاً یک‌بار» وجود ندارد
تاریخچه	Room به‌همراه Paging و `RemoteMediator`	بارگذاری همهٔ پیام‌ها در حافظه
مدیا	اول آپلود قابل ازسرگیری، بعد ارسال شناسهٔ مدیا	Base64 داخل پیام

تلهٔ مصاحبه

«سوکت را در یک foreground service زنده نگه دارید تا پیام‌ها در background برسند.» — باتری را تخلیه می‌کند، و از Android 14 به بعد، هر foreground service باید type با یک کار قابل‌مشاهده برای کاربر جور باشد — یک listener بیکار چنین کاری نیست. از پیام‌های high-priority در FCM استفاده کنید که به یک نوتیفیکیشن قابل‌مشاهده منجر می‌شوند.

سؤال تکمیلی

«رمزنگاری سرتاسری اضافه کنیم؟» — سرور دیگر نمی‌تواند پیام را بخواند، جست‌وجو کند یا پیش‌نمایش بسازد؛ پوش فقط یک شناسه حمل می‌کند و دستگاه قبل از نمایش نوتیفیکیشن رمزگشایی می‌کند. مدیریت کلید و پشتیبانی از چند دستگاه، بخش سخت ماجرا می‌شوند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. طراحی فید offline-first — نیازمندی‌ها و API

کارکردی: اسکرول بی‌پایان، pull to refresh، لایک و ذخیره، تصویر، باز شدن فوری با آخرین محتوا. **غیرکارکردی:** محتوا در همان فریم اول از کش، اسکرول روان، مصرف دادهٔ کم، بدون تکرار هنگام صفحه‌بندی.

```
GET /v1/feed?cursor=<opaque>&limit=20
→ { "items": [...], "nextCursor": "c_8f3a" }

PUT    /v1/posts/{id}/like        idempotent set-state
DELETE /v1/posts/{id}/like
```

تلهٔ مصاحبه

«*با* `?page=3` *صفحه‌بندی کنید.*» — در یک فید زنده، پست‌های جدید offsetها را جابه‌جا می‌کنند: کاربر **آیتم تکراری و آیتم جاافتاده** می‌بیند. از cursorهای مبهم (opaque) که سرور صادر می‌کند استفاده کنید.

تلهٔ مصاحبه

«*یک endpoint مثل* `POST /posts/{id}/toggle-like` *بسازید.*» — یک دابل‌تپ به‌علاوهٔ یک retry، لایک را دو بار برمی‌گرداند. **state** را مدل کنید، با `PUT` و `DELETE` که طبق تعریف idempotent هستند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. Room منبع است، Paging پنجره

```
@OptIn(ExperimentalPagingApi::class)
class FeedRepository(
    private val db: FeedDatabase,
    private val api: FeedApi,
) {
    fun feed(): Flow<PagingData<Post>> = Pager(
        config = PagingConfig(pageSize = 20),
        remoteMediator = FeedRemoteMediator(db, api),
        pagingSourceFactory = { db.postDao().pagingSource() },
    ).flow.map { page -> page.map { it.toDomain() } }
}

// In FeedRemoteMediator: don't refresh while cache is fresh
override suspend fun initialize(): InitializeAction =
    if (cacheIsFresh()) InitializeAction.SKIP_INITIAL_REFRESH
    else InitializeAction.LAUNCH_INITIAL_REFRESH
```

- `RemoteMediator` هر صفحه و cursor آن را در **یک** تراکنش Room می‌نویسد؛ UI فقط از Room می‌خواند
- `cachedIn(viewModelScope)` در ViewModel داده‌های صفحه‌بندی‌شده را هنگام چرخش صفحه نگه می‌دارد (جزئیات mediator در روز ۱۴)



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. نوشتن و مقیاس روی دستگاه

- **لایک:** به‌روزرسانی خوش‌بینانهٔ محلی به‌علاوهٔ outbox؛ سرور تعداد نهایی را برمی‌گرداند
- **تصویر:** نسخه‌ای با اندازهٔ مناسب از CDN بخواهید؛ کش حافظه و دیسک؛ prefetch صفحهٔ بعد
- **ذخیره‌سازی:** برای کش سقف بگذارید (مثلاً N پست آخر یا ۷ روز اخیر) و هنگام refresh پاک‌سازی کنید
- **تازگی:** به‌جای جابه‌جا کردن لیست زیر انگشت کاربر، یک دکمهٔ «پست‌های جدید» نشان دهید
- **رتبه‌بندی** روی سرور انجام می‌شود؛ کلاینت همان ترتیبی را که دریافت می‌کند رندر می‌کند

تلهٔ مصاحبه

«*ViewModel فید را در یک لیست نگه می‌دارد.*» — با process death از بین می‌رود، دو منبع حقیقت می‌سازد، و حافظه با هر صفحه بزرگ‌تر می‌شود. Room به‌همراه Paging یک پنجرهٔ محدود نگه می‌دارد.

سؤال تکمیلی

«*پستی روی سرور حذف شده — چطور از اپ ناپدید می‌شود؟*» — با tombstoneها در یک delta sync که با `since=cursor` کار می‌کند، یا صفحهٔ refreshشده ردیف‌های قدیمی را جایگزین می‌کند؛ باز کردن آن پست هم `404` یا `410` را بدون crash مدیریت می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

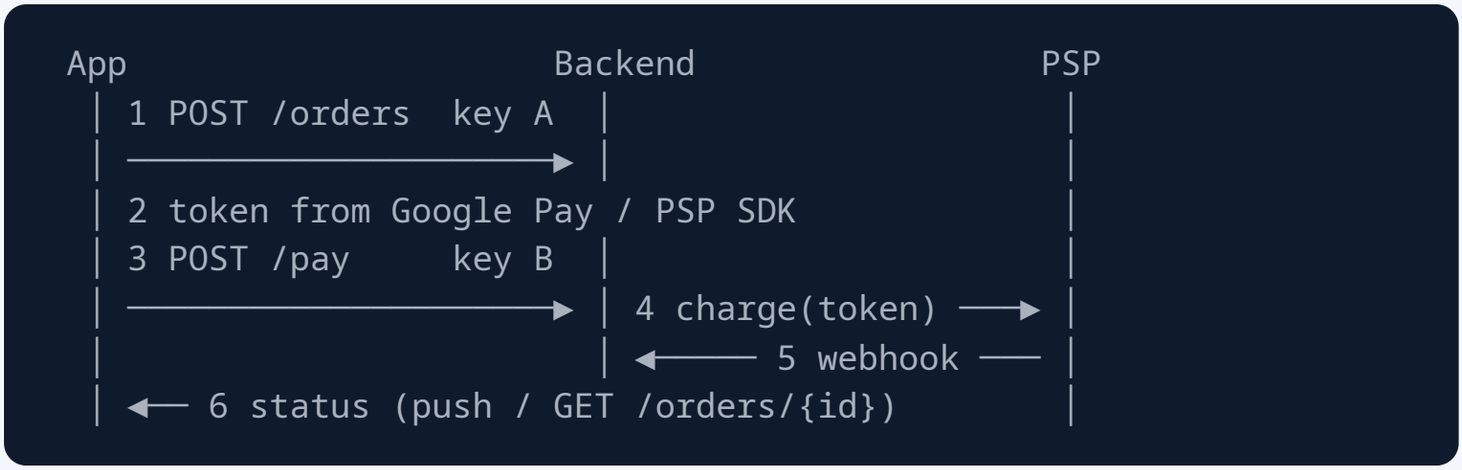
طراحی پرداخت، gateway و BFF و نسخه‌بندی، مقیاس و observability و trade-offها



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. طراحی جریان پرداخت — نیازمندی‌ها و اعتماد

- **هرگز دو بار شارژ نکنید، هرگز نتیجه را گم نکنید** — حتی با process death یا قطع شبکه وسط درخواست
- اطلاعات کارت هرگز به سرورها یا لاگ‌های شما نمی‌رسد: Google Pay یا SDK ارائه‌دهنده یک **توکن** برمی‌گرداند
- **سرور** منبع حقیقت است؛ کلاینت قابل دستکاری است



- **PSP** یعنی ارائه‌دهندهٔ خدمات پرداخت (مثل Stripe یا Adyen).
  webhook آن به بک‌اند شما حرف آخر را می‌زند، نه callback اپ.
- دو قصد کاربر، دو idempotency key: کلید A برای «ساخت سفارش»، کلید B برای «پرداخت این سفارش».



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. پرداخت یک state machine است

```
CREATED → SUBMITTING ─┬─► SUCCEEDED
                      ├─► REQUIRES_ACTION (3DS) → …
                      ├─► FAILED (declined)
                      └─► UNKNOWN (timeout, crash)
                              → reconcile: GET status
```

```kotlin
suspend fun pay(orderId: String, token: String): PaymentStatus {
    val attempt = dao.openAttempt(orderId)
        ?: PaymentAttempt(
            idempotencyKey = UUID.randomUUID().toString(),
            orderId = orderId,
            status = PaymentStatus.SUBMITTING,
        ).also { dao.insert(it) }      // persist before sending
    val status = try {
        api.pay(orderId, token, attempt.idempotencyKey).status
    } catch (e: IOException) {
        PaymentStatus.UNKNOWN           // NOT failed: reconcile
    }
    dao.updateStatus(attempt.idempotencyKey, status)
    return status
}
```

- `openAttempt` تلاش نیمه‌تمام همین سفارش را برمی‌گرداند، پس retry بعد از crash یا timeout **از همان key دوباره استفاده می‌کند**
- وضعیت `UNKNOWN` فرایند تطبیق (reconcile) با `GET /orders/{id}` را شروع می‌کند، هرگز یک صفحهٔ خودکار «ناموفق» را



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

«*درخواست timeout شد، پس پیام پرداخت ناموفق را نشان بده.*» — شاید شارژ انجام شده باشد. timeout یعنی **UNKNOWN**: پیام «در حال تأیید پرداخت...» را نشان دهید، وضعیت سفارش را بپرسید، و بگذارید هر retry از **همان** idempotency key استفاده کند.

تلهٔ مصاحبه

«*callback مربوط به SDK می‌گوید موفق بود، پس محتوا را باز کن.*» — کلاینت قابل اعتماد نیست. فقط وقتی باز کنید که **سرور** تأیید کند (webhookهای ارائه‌دهنده، تأیید سمت سرور).

- احراز هویت 3DS کاربر را به اپ بانک می‌فرستد؛ ممکن است process شما در این فاصله از بین برود. در بازگشت، تلاش ذخیره‌شده را بازیابی و reconcile کنید.
- نتیجهٔ نهایی را به شکل **state** نشان دهید، نه یک event یک‌باره (روز ۳).

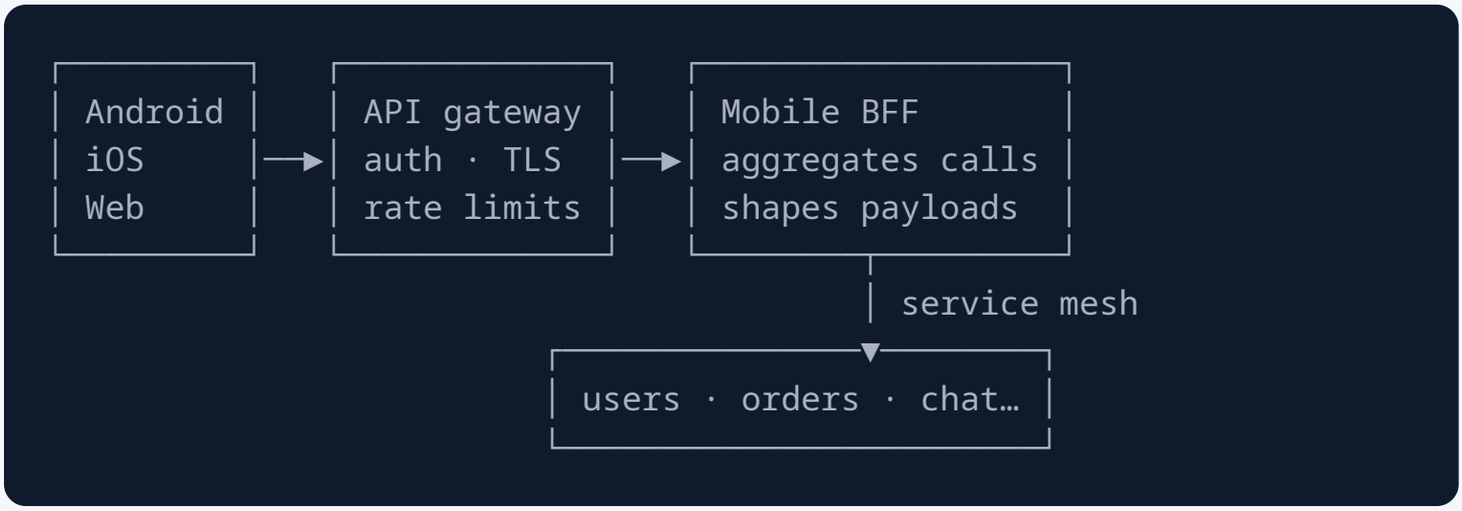
سؤال تکمیلی

«*کالای دیجیتال چطور؟*» — در اپی که از Google Play توزیع می‌شود، معمولاً باید از Google Play Billing استفاده کنید (در برخی مناطق جایگزین وجود دارد). توکن خرید را روی سرور خودتان تأیید و خرید را acknowledge کنید — خریدهای acknowledgeنشده خودکار بازپرداخت می‌شوند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. یکپارچه‌سازی در مقیاس: gateway، BFF و service mesh



- **API gateway:** یک نقطهٔ ورود عمومی — احراز هویت، TLS، محدودیت نرخ، مسیریابی
- **BFF:** یک API که برای یک نوع کلاینت شکل گرفته — در حالت ایده‌آل یک فراخوانی برای هر صفحه، و متعلق به تیم موبایل یا نزدیک به آن
- **service mesh:** فقط ترافیک سرویس به سرویس؛ اپ هرگز آن را نمی‌بیند (این یکی را نسخهٔ ۱ درست گفته بود)

تلهٔ مصاحبه

«*اپ مستقیم با دوازده microservice حرف می‌زند.*» — رفت‌وبرگشت‌های زیاد با تأخیر شبکهٔ موبایل، همهٔ سرویس‌های داخلی در معرض دسترسی عمومی، و اپ با هر تغییر ساختار بک‌اند می‌شکند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. نسخه‌بندی وقتی کلاینت‌ها آپدیت نمی‌شوند

نسخه‌های قدیمی اپ **سال‌ها** نصب می‌مانند. سرور باید با نسخه‌های قبلی سازگار بماند:

```
val json = Json {
    ignoreUnknownKeys = true  // a newer server adds fields
    coerceInputValues = true  // unknown enum value -> default
}

enum class OrderStatus { PENDING, PAID, SHIPPED, UNKNOWN }

@Serializable
data class OrderDto(
    val id: String,
    val status: OrderStatus = OrderStatus.UNKNOWN,
)
```

- **فقط تغییرات افزایشی؛** هیچ فیلدی را تغییر نام ندهید یا برای معنای دیگری استفاده نکنید
- چک تغییرات شکننده در CI: مقایسهٔ OpenAPI، چک schema در GraphQL، `buf breaking`؛ و تست قرارداد با Pact
- آپدیت اجباری برای نسخه‌های زیر حداقل نسخهٔ پشتیبانی‌شده (روز ۲۴) — آخرین راه‌حل

تلهٔ مصاحبه

یک `enum` ساده با `Json` پیش‌فرض: مقدار جدیدی مثل `"REFUNDED"` در همهٔ کلاینت‌های قدیمی یک `SerializationException` پرتاب می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. مقیاس و observability در کلاینت

دغدغه	پاسخ طراحی
باتری	کار دسته‌ای، constraintهای WorkManager، بدون سوکت در background
مصرف داده	delta sync با `since=cursor`، فشرده‌سازی، نسخه‌های مختلف تصویر
ذخیره‌سازی	سقف و پاک‌سازی کش؛ migrationهای تست‌شده
دستگاه ضعیف	Paging، کوچک‌سازی تصویر، فیچر بر اساس ردهٔ دستگاه
اثر ناوگان	زمان‌بندی با jitter، `Retry-After`، kill switch، rollout مرحله‌ای

برای خود طراحی متریک تعریف کنید (observability):

- چت: تأخیر ارسال تا ack، پیام‌های گیرکرده در `PENDING`، نرخ reconnect
- فید: زمان تا اولین محتوا، jank اسکرول، نرخ hit کش
- پرداخت: سهم نتیجه‌های `UNKNOWN`، زمان تا reconcile
- هدر `traceparent` استاندارد W3C را بفرستید تا spanهای اپ به traceهای بک‌اند وصل شوند (OpenTelemetry)؛ بدون PII در لاگ‌ها



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. جمع‌بندی مصاحبه: trade-offها

سؤال	گزینهٔ الف	گزینهٔ ب
لحظه‌ای بودن	WebSocket: تازه، پرهزینه	پوش و sync: ارزان، با تأخیر
تعارض	LWW: ساده، داده گم می‌کند	نسخه یا ادغام: امن، کار بیشتر
API	REST با BFF: قابل کش	GraphQL: منعطف، کش سخت‌تر
UI	خوش‌بینانه: سریع، نیازمند rollback	بدبینانه: کند، قطعی
کش	اول کش: فوری، شاید کهنه	اول شبکه: تازه، ضعیف در آفلاین

تلهٔ مصاحبه

«*فقط یک طراحی درست وجود دارد.*» — مصاحبه‌گر به استدلال نمره می‌دهد. فرض‌هایتان را بگویید، و وقتی یک نیازمندی را عوض می‌کنند، **طراحی را بلند بلند عوض کنید.**

جواب سینیور در یک جمله

«برای هر تصمیم، نیازمندی‌ای را که آن را تحمیل می‌کند نام ببر، بگو چه چیزی را فدا کردی، و بگو در production از کجا می‌فهمی که اشتباه کرده‌ای.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۳۰ — خلاصهٔ یک‌صفحه‌ای

- چارچوب: نیازمندی‌ها ← قرارداد API ← داده و ذخیره‌سازی ← sync و آفلاین ← UI state ← مقیاس ← observability ← و در آخر trade-offها
- بک‌اند یک جعبه پشت یک قرارداد است؛ در سمت دستگاه عمیق شوید.
- Room منبع واحد حقیقت است؛ شبکه در آن می‌نویسد و صفحه‌ها `Flow` را observe می‌کنند.
- retry فقط برای خطاهای گذرا، با backoff و jitter، با رعایت `Retry-After`، و فقط در **یک** لایه.
- «دقیقاً یک‌بار» در شبکه وجود ندارد: حداقل یک‌بار، به‌علاوهٔ idempotency key و حذف تکراری‌ها.
- تعارض: نسخه‌های سرور (`If-Match` با ETag) بهتر از LWW با ساعت دستگاه است.
- لحظه‌ای بودن: وقتی UI دیده می‌شود WebSocket؛ در background، پوش FCM فقط یک سیگنال است و بعد sync.
- مرتب‌سازی با sequence سرور، هرگز با timestamp دستگاه. صفحه‌بندی با cursor، نه offset.
- پرداخت: timeout یعنی UNKNOWN و باید reconcile شود؛ سرور تأیید می‌کند، هرگز کلاینت.
- نسخه‌های قدیمی اپ سال‌ها زنده‌اند: تغییرات افزایشی در API، مقدار پیش‌فرض برای enum ناشناخته، چک قرارداد در CI.



اسماعیل مرادی
@esmaeeil-moradi700

۳۰ روز در یک اسلاید

روزها	چه چیزهایی را پوشش دادیم
۱ تا ۳	چرخهٔ حیات و process death، Fragment و Navigation، معماری
۴ و ۵	Coroutines و Flow
۶ تا ۸	ملزومات Jetpack، تزریق وابستگی، Compose
۹ و ۱۰	تست: unit، UI، اسکرین‌شات، بنچمارک
۱۱ و ۱۲	شبکه، امنیت شبکه و احراز هویت
۱۳ و ۱۴	لایهٔ data، offline-first و sync
۱۵ و ۱۶	کار پس‌زمینه، سرویس‌ها، آلارم و پوش
۱۷ و ۱۸	پرفورمنس: jank، حافظه، حجم اپ، باتری
۱۹ و ۲۰	امنیت: ذخیره‌سازی، Play Integrity، passkey
۲۱ و ۲۲	Gradle و پرفورمنس build
۲۳ و ۲۴	CI/CD، انتشار، rollout و feature flag
۲۵ و ۲۶	ماژولار کردن، Kotlin پیشرفته
۲۷ تا ۲۹	معماری، پرفورمنس و مقیاس جهانی
۳۰	طراحی سیستم و یکپارچه‌سازی



اسماعیل مرادی

@esmaeeil-moradi700

ممنونم

نسخهٔ ۲ به خاطر شما وجود دارد. بعد از نسخهٔ ۱ از من خواستید اسلایدها کمتر و عمیق‌تر باشند، به‌جای تئوری کد واقعی ببینید، تله‌هایی را بشناسید که مصاحبه‌گرها واقعاً رویشان دست می‌گذارند، و محتوا با اندروید امروز به‌روز شود. تک‌تک روزهای این نسخه بر اساس همین بازخوردها از نو ساخته شد.

به همهٔ کسانی که سی روز نسخهٔ ۲ را دنبال کردند — پستی را ذخیره کردند، در کامنت‌ها به سؤال‌های خودسنجی جواب دادند، با یک تله بحث کردند، یا کاروسلی را برای دوستی که برای مصاحبه آماده می‌شد فرستادند: **ممنونم.** این چالش مال همهٔ ما بود، نه فقط من.

یک توصیهٔ آخر

این جواب‌ها را حفظ نکنید. آن‌قدر خوب بفهمیدشان که بتوانید از اصول اولیه دوباره بسازیدشان — این همان چیزی است که مصاحبه‌گرها واقعاً دنبالش هستند.



اسماعیل مرادی
@esmaeeil-moradi700

بیایید در ارتباط بمانیم

- **کدام روز بیشتر از همه به شما کمک کرد** — و کدام موضوع ارزش یک بررسی عمیق‌تر را دارد؟ در کامنت‌ها بنویسید.
- **برای مصاحبهٔ طراحی سیستم آماده می‌شوید؟** اپی را که دوست دارید طراحی‌اش را ببینید در کامنت بنویسید.
- **کل مجموعه را ذخیره کنید** — هر ۳۰ روز، به فارسی و انگلیسی، در پروفایل من هست.
- **در لینکدین با من در ارتباط باشید** و خبر موفقیت‌هایتان در مصاحبه را با من در میان بگذارید — همهٔ پیام‌ها را می‌خوانم.

به یاد گرفتن و ساختن ادامه دهید — و در مصاحبهٔ بعدی‌تان موفق باشید.



اسماعیل مرادی
@esmaeeil-moradi700